Message Logging
(basic message logging by clicking **Treat**, which runs console.log(...) with a message plus a stats object. This shows normal logs and how objects can be expanded in the Console.)

Log Info
(info-level logging by clicking **Play**, which runs console.info(...). This creates an “info” message that can be filtered by level in DevTools.)

Log Warning
(warning-level logging by clicking **Exercise**, which runs console.warn(...). This produces a warning entry (yellow) that stands out and can be filtered by level.)

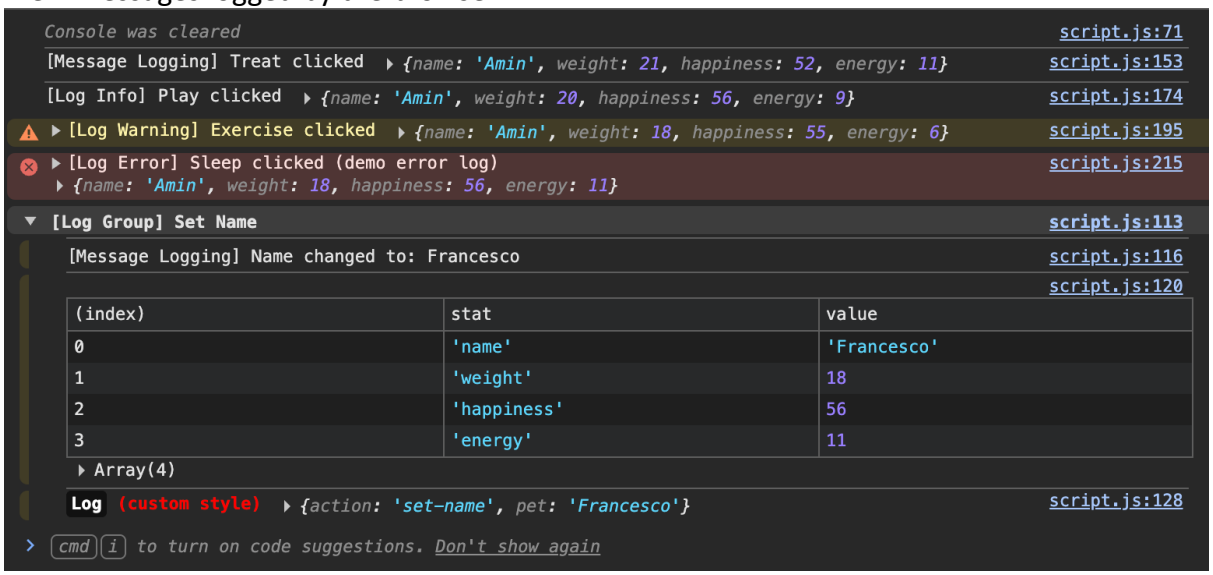
Log Error
(error-level logging by clicking **Sleep**, which runs console.error(...). This logs an error message (red) without crashing the app.)

Log Table
(table logging by clicking **Set Name**, which calls console.table([...]) to display pet stats in a table format.)

Log Group
(grouping by clicking **Set Name**, uses console.groupCollapsed(...) and console.groupEnd() to place multiple related logs into one collapsible group in the Console.

Log Custom
(custom styled logging by clicking **Set Name**, which uses a %c formatted console.log(...) with CSS strings to style the log text in the Console.)

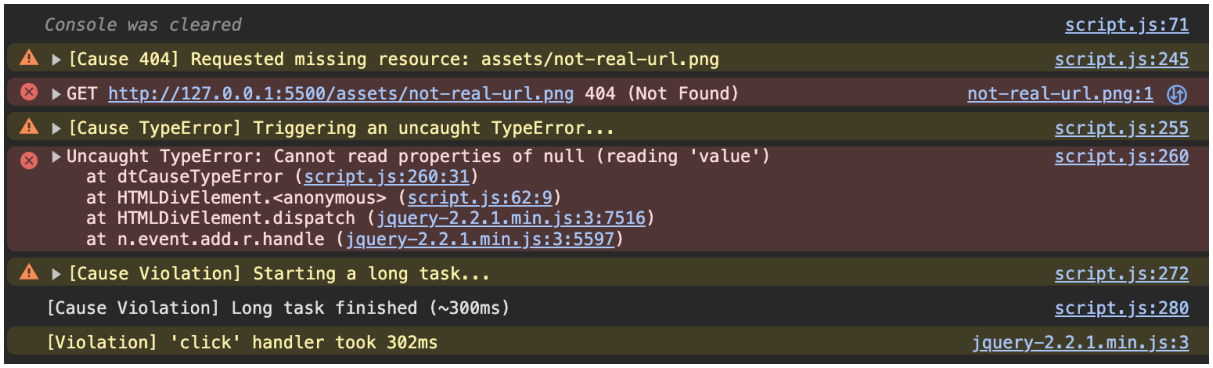
View messages logged by the browser



Cause 404 network error
by clicking **Cause 404**, which creates a JavaScript Image() and sets img.src to a file path that does not exist (ex: assets/not-real-url.png). The browser requests it, producing a **404** entry in the Network tab.

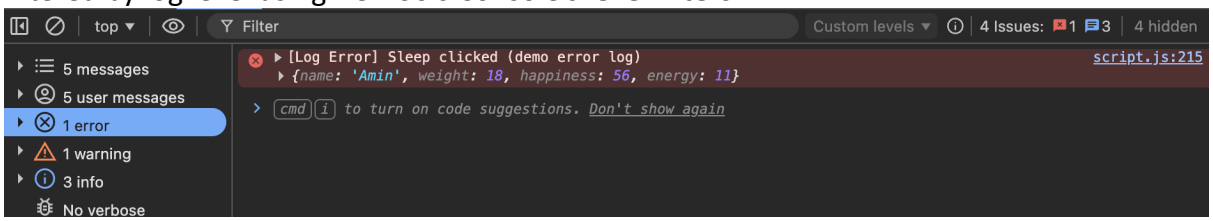
Cause TypeError
an uncaught TypeError by clicking **Cause TypeError**, which intentionally tries to access a property on null (ex: nothingHere.value). This throws an **Uncaught TypeError** and shows a stack trace in the Console pointing to the line in script.js.

Cause Violation
violation by clicking **Cause Violation**, which runs an intentional busy loop (blocking the main thread for around 300ms). Chrome prints a [Violation] message in the Console when a task takes too long.

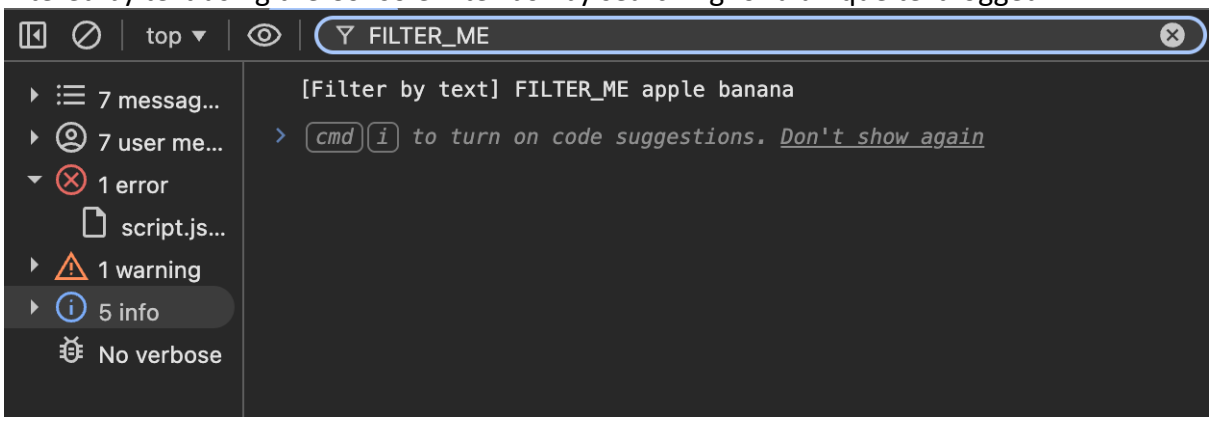


Filter Messages
generated a set of console messages on page load using a helper function (dtGenerateFilterLogs()), so that there are consistent logs ready to be filtered in DevTools.

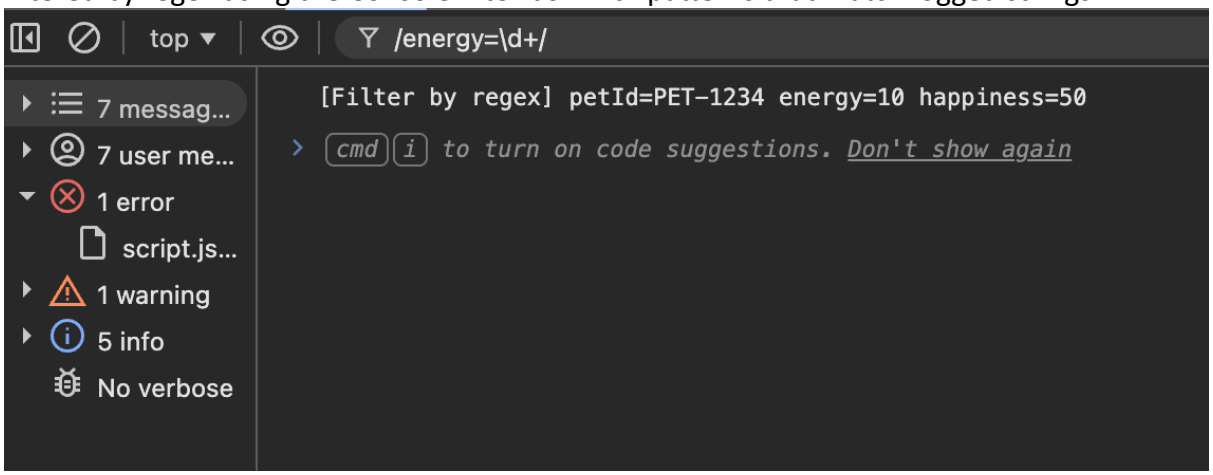
Filter by log level
filtered by log level using DevTools Console's **level filters**.



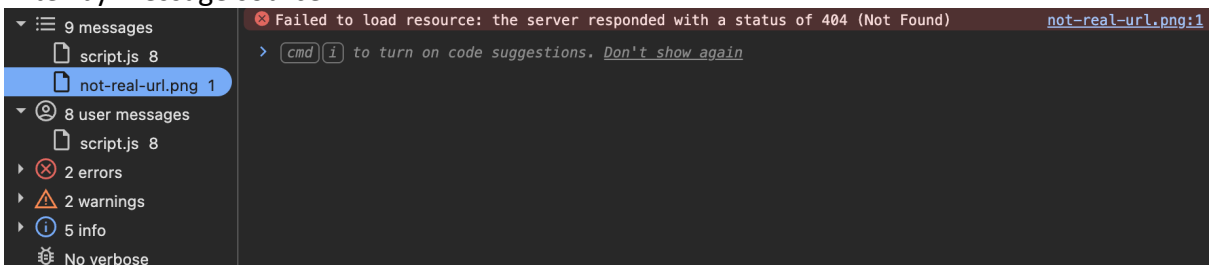
Filter by text
filtered by text using the Console filter box by searching for a unique text logged



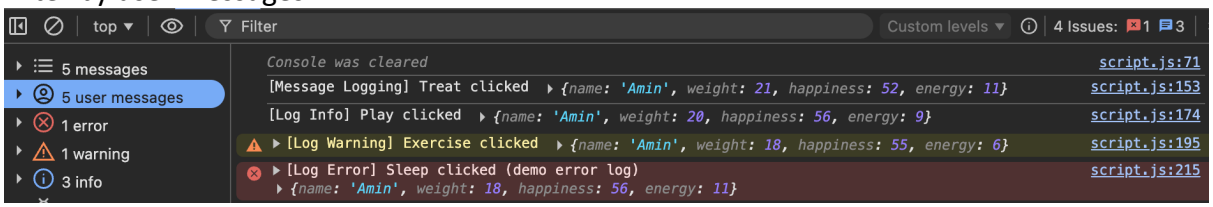
Filter by regular expression
filtered by regex using the Console filter box with patterns that match logged strings



Filter by message source

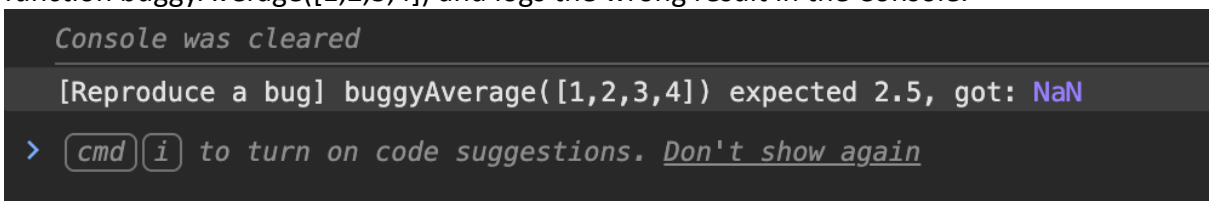


Filter by user messages



Reproduce a bug

reproduced a bug by clicking **Reproduce Bug**, which calls an intentionally broken function buggyAverage([1,2,3,4]) and logs the wrong result in the Console.



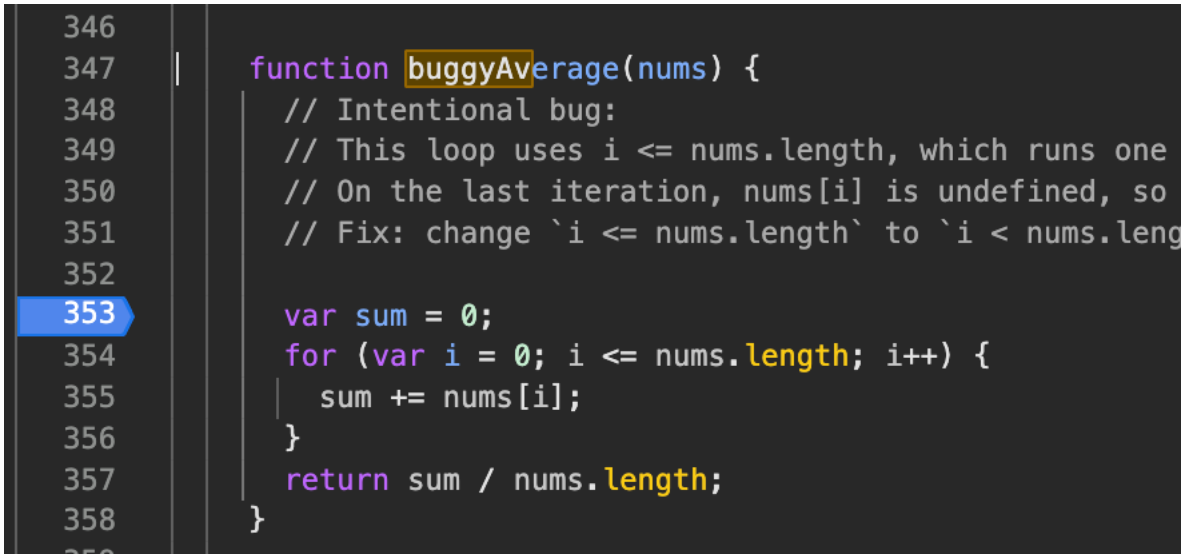
Get Familiar with the Sources UI

used the **Sources** tab to open script.js, locate the bug function, and navigate code.

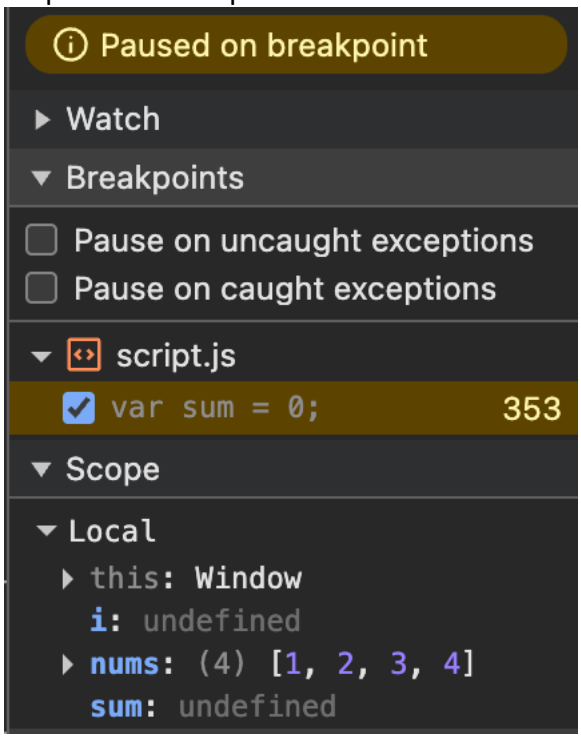
Pause the code with a breakpoint

Set a line-of-code breakpoint

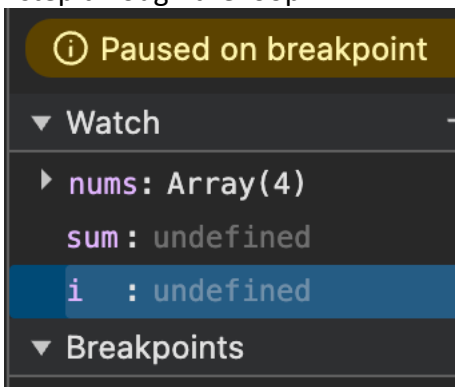
paused execution by setting a breakpoint inside buggyAverage() and then clicking **Reproduce Bug** again. Execution stops at the breakpoint and highlights the current line.



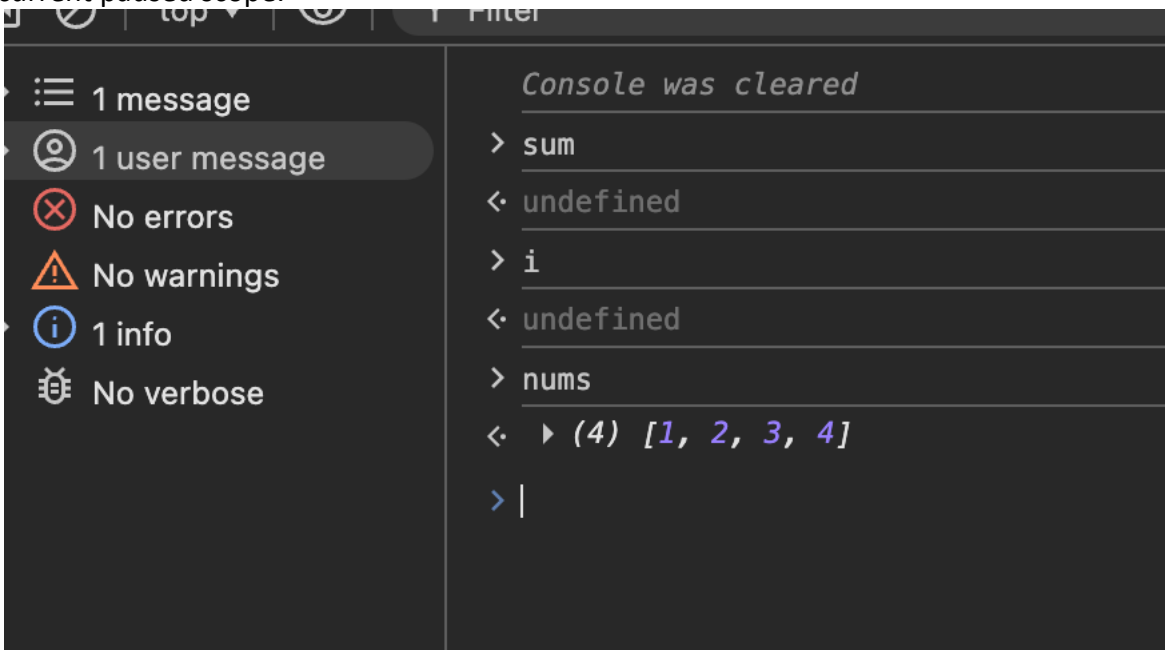
Check variable values
The Scope Pane
used the Scope pane to inspect local function variables (nums, i, sum) and confirm that the loop runs one step too far and hits undefined.



Watch Expressions
added watch expressions: i, sum, and nums so DevTools continuously displays their values as I step through the loop.



The Console
While paused at the breakpoint, I used the DevTools Console to evaluate expressions in the current paused scope.



Apply a fix (<= to < in condition)
fixed the bug by editing the loop condition from i <= nums.length to i < nums.length (so it stops before the out-of-bounds index).

```
var sum = 0;
for (var i = 0; i < nums.length; i++) {
  sum += nums[i];
}
return sum / nums.length;
}
```